

第二十一届中国研究生电子设计竞赛

技术论文

论文题目：

面向飞腾平台的 TCP/CAN 双模 UDS 诊断
系统设计与 CAN 驱动性能分析

TCP/CAN Dual-Mode UDS Diagnostic
System for the Phytium Platform

参赛单位：北华大学

队伍名称：你别过来呀

指导老师：王丹

参赛队员：胡安、张泽浩

完成时间：2026 年 5 月 18 日

摘 要

随着智能网联汽车技术的发展，车载诊断通信对嵌入式平台的实时性和可靠性提出了更高要求。本文面向飞腾 E2000 嵌入式平台，设计并实现了一套基于 Qt 的 TCP/CAN 双模 UDS (Unified Diagnostic Services, ISO 14229-1) 诊断与压测系统。系统采用客户端-服务端架构：PC 客户端基于 Qt 6.8 框架搭建，支持 TCP 和 CAN 双通道通信，实现了 5 项核心 UDS 诊断服务（诊断会话控制、安全访问、读数据标识符、ECU 复位、测试者存在），并内置阶梯加压式压力测试模块；服务端部署于飞腾派，采用多线程模型（CAN/TCP/UDS 线程）并通过 CPU 绑定和中断亲和性进行优化。在压力测试过程中，系统在 300 req/s 以上出现 ENOBUFS 错误和 CAN 总线丢包现象。本文通过双 USB-CAN 压测排除应用层缺陷，通过 cangen 纯 TX 测试排除硬件缺陷，结合飞腾 Linux 内核驱动源码分析（drivers/net/can/phytium/），推测瓶颈为板载 CAN 在双向 UDS 全栈负载下的 TX 调度效率不足，具体表现为单帧入队、硬件 FIFO 未充分利用、中断唤醒延迟。在不修改内核驱动的前提下，实施了 CPU 线程绑定、IRQ 中断亲和性、用户态环形缓冲区及 ISO-TP 协议栈内联优化，使 300 req/s 负载下的丢包率由 13.86% 降至 2.73%（降幅 80.3%），50 req/s 演示负载下丢包率仅为 0.58%。本文的工作作为飞腾平台 CAN 子系统的性能标定和驱动优化提供了工程参考。

关键词：飞腾平台；CAN 总线；UDS 诊断；驱动性能分析；车载通信

Abstract

With the development of intelligent connected vehicle technology, on-board diagnostic communication imposes higher requirements on the real-time performance and reliability of embedded platforms. This paper presents a TCP/CAN dual-mode UDS (ISO 14229-1) diagnostic and stress-testing system designed for the Phytium E2000 embedded platform. The client, built on Qt 6.8, supports dual-channel communication via TCP and CAN, implements five core UDS services, and integrates a stepped-load stress testing module. The server is deployed on the Phytium Pi with a three-thread model (CAN/TCP/UDS) optimized through CPU affinity binding and interrupt affinity tuning.

During stress testing at 300+ requests per second, ENOBUFS errors and CAN frame loss were observed. Through cangen isolation testing, application-layer causes were ruled out. Source-level analysis of the Phytium Linux kernel CAN driver (drivers/net/can/phytium/) suggests that the bottleneck lies in insufficient TX scheduling efficiency of on-board CAN under bidirectional UDS full-stack load, characterized by single-frame enqueue, underutilization of hardware FIFO, and delayed interrupt wakeup. Without modifying the kernel driver, optimizations including CPU core binding, IRQ affinity tuning, user-space ring buffering, and inline ISO-TP polling were implemented, reducing the frame loss rate at 300 req/s from 13.86% to 2.73% (an 80.3% improvement), and to 0.58% at 50 req/s. This work provides an engineering reference for performance characterization and driver optimization of the CAN subsystem on Phytium platforms.

Keywords: Phytium Platform; CAN Bus; UDS Diagnostics; Driver Performance Analysis; In-Vehicle Communication

目 录

一、作品难点与创新	1
1.1 难点分析	1
1.2 创新点	2
二、方案论证与设计	4
2.1 系统总体方案	4
2.2 系统架构设计	5
三、原理分析与硬件电路图	7
3.1 CAN 总线通信原理	7
3.2 phytiun_can 硬件特性	7
3.3 驱动源码分析	7
3.4 硬件连接图	10
四、软件设计与流程	11
4.1 客户端软件设计	11
4.2 服务端软件设计	11
4.3 UDS 请求处理流程	12
4.4 数据持久化与日志系统	13
五、系统测试与分析	15
5.1 测试环境与方案	15
5.2 测试结果	15
5.3 驱动瓶颈验证	20
六、总结	22
参考文献	23

一、作品难点与创新

1.1 难点分析

本作品在工程实现、性能优化和系统分析三个层面面临以下核心技术难点：

（1）双模异构通信链路的实时性与可靠性平衡。系统同时支持 TCP 和 CAN 两条物理链路，两者在传输速率（TCP 可达 Mbps 级 vs CAN 500 kbps）、帧格式（TCP 流式 vs CAN 定长 8 字节）、错误处理机制（TCP 内核重传 vs CAN 硬件仲裁与应答）上存在本质差异。TCP 因流式传输需处理粘包拆包，自定义二进制帧协议引入魔数同步和 CRC16 校验开销；CAN 因 8 字节有效载荷限制需要 ISO-TP 协议进行多帧分段与重组，增加了传输延迟和实现复杂度。在双链路同时承载 UDS 诊断业务的场景下，需保证跨通道的请求-响应时序一致性和错误语义的统一处理。

（2）高负载下 CAN 总线丢包问题的多层级定位与交叉验证。在阶梯加压至 300 req/s 以上时，系统出现 ENOBUFS 错误和丢包率上升。该问题涉及多个可能的层级：Qt 端 USB-CAN 适配器初始化模式（是否正常发送 ACK）、应用层 socket 配置（阻塞/非阻塞、环形缓冲设计）、内核 SocketCAN 子系统

（SO_SNDBUF 队列大小、中断唤醒路径）、以及 phytiun_can 驱动层（TX FIFO 填充策略）。本文通过三组对照实验——双 USB-CAN 压测（排除应用层代码缺陷）、cangen 纯 TX 测试（排除硬件缺陷）、以及板载 CAN 与 USB-CAN 的对比压测（指向板载 CAN 特定场景）——逐步缩小问题范围，最终将瓶颈范围缩小至板载 CAN 在双向全栈 UDS 负载下的调度效率。

（3）SoC 集成 CAN 控制器的性能天花板确认与工程标定。飞腾 E2000 phytiun_can 控制器在设备树中声明 64 槽 TX/RX FIFO，支持 CAN FD，但实际高负载测试中性能远低于预期。控制器作为 SoC 内置外设，其性能直接反映飞腾 CPU 的中断响应能力和驱动调度效率，而非独立 CAN 芯片的硬件能力。如何在缺乏完整技术文档的情况下，通过源码阅读和寄存器级别分析来确认硬件能力与驱动

利用之间的差距，并通过量化实验标定出可工程使用的性能边界，是本工作的核心难点。

1.2 创新点

本作品的创新性不在于单一技术点的突破，而聚焦于国产嵌入式平台 CAN 性能分析与应用层优化，形成一套可复用的工程方法论，建立了一套完整的“系统异常发现→内核层隔离验证→驱动源码追溯→应用层优化兜底→性能量化标定”的系统级问题分析方法论。具体体现在以下三个方面：

提出“多维度交叉验证 + 逐层排除”的系统级瓶颈定位方法。区别于常规的压测-调参模式，本文在发现 ENOBUFS 异常后，依次通过三个维度进行排除：第一，使用双 USB-CAN 替代板载 CAN 执行同参数 UDS 压测（2000 req/s 丢包仅 0.02%），排除应用层代码缺陷；第二，通过 cangen 内核工具在板载 CAN 上执行纯 TX 发包测试（1200 fps 正常），排除硬件缺陷，并将问题范围缩小至双向全栈负载场景；第三，结合 phytium_can 驱动源码分析（start_xmit 单 skb 架构、netif_stop_queue 机制），为“为何双向负载下中断延迟影响被放大”提供技术解释。该方法可推广至国产 SoC 嵌入式平台的性能瓶颈定位场景。

提出“不修改内核驱动前提下的应用层异步发送架构”。在确认驱动瓶颈后，受客观条件限制无法修改和重新编译内核驱动。本文在应用层设计了一套三层优化方案：第一层，通过 pthread_setaffinity_np 和 /proc/irq/N/smp_affinity 将 CAN 线程与 CAN 中断绑定至同一 CPU 核心，将中断响应延迟从 $>10\mu s$ 压缩至 $<2\mu s$ ；第二层，设计 512 槽用户态 TX 环形缓冲区，结合 QSocketNotifier

::Write 的 POLLOUT 事件驱动异步出队，使发送速率自然匹配硬件 FIFO 的排空节奏；第三层，将 ISO-TP 协议栈的 poll 机制从 QTimer 定时器驱动改为 send/receive 后立即内联调用，消除 1~10ms 的调度延迟。该方案在应用层最大限度释放了硬件潜力，为此类“无法改驱动但需要优化性能”的工程场景提供了参考。

建立 phytium_can 在 UDS 诊断负载下的量化性能模型。通过 6 组对照实验（板载 CAN vs USB-CAN、绑核 vs 未绑核、飞腾派 vs 香橙派、50/300/600/2000 req/s 四级负载），定量标定了板载 CAN 的稳定工作区域（ ≤ 300 req/s，丢包

<3%)、性能退化区域 (300-600 req/s) 和不可用区域 (>600 req/s)。该模型不仅为本次系统设计提供了参数依据,更为后续所有基于飞腾平台 CAN 通信的应用开发提供了可直接参考的工程数据。

二、方案论证与设计

2.1 系统总体方案

本系统的设计目标是构建一套面向飞腾嵌入式平台的 TCP/CAN 双模 UDS 诊断通信系统，并在阶梯加压式压力测试中验证和优化通信性能。系统的核心应用场景为车载诊断——PC 端诊断仪通过 CAN 总线直接连接 ECU（服务端），或通过 TCP/IP 网络进行远程诊断。系统采用客户端-服务端架构，PC 端为 Qt 上位机客户端，飞腾派为嵌入式服务端，两者通过 TCP/IP 网络和 CAN 总线两条物理链路互联，构成完整的车载诊断通信闭环。

方案选型依据如下。

第一，选择 C/S 架构而非单体架构，是因为诊断场景天然涉及诊断仪（客户端）与 ECU（服务端）的物理分离。客户端需同时支持 TCP 和 CAN 两种通道以适应不同部署环境——CAN 通道用于本地直连诊断（低延迟、高可靠），TCP 通道用于远程诊断（灵活部署、不受距离限制）。

第二，选择 Qt 6.8 作为开发框架。Qt 成熟的信号槽机制天然适配多线程通信场景，跨线程 QueuedConnection 无需手动加锁即可实现线程安全的数据传递。QSocketNotifier 可直接对接 Linux SocketCAN 的文件描述符事件驱动模型，避免轮询开销。Qt Charts 模块可快速搭建实时压测的动态曲线界面。此外，Qt 的跨平台特性保证客户端可在 Windows 和 Linux 双平台下编译运行，不依赖特定操作系统。

第三，选择 ISO 15765-2（ISO-TP）作为 CAN 传输层协议。UDS 诊断消息的长度可达 4095 字节，远超经典 CAN 帧的 8 字节有效载荷限制。ISO-TP 定义了四种帧类型——单帧（SF， ≤ 7 字节）、首帧（FF，声明总长度并携带前 6 字节）、连续帧（CF，序列号 + 7 字节数据）和流控帧（FC，控制发送节奏与块大小），通过分段传输与流控机制实现了 CAN 总线上的大数据量可靠传输。本文使用开源 C 语言 ISO-TP 协议栈，将其封装为 Qt 可调用的接口，实现 CAN 收发与 UDS 协议层之间的桥接。

第四，TCP 通道采用自定义二进制帧协议替代传统的 HTTP/JSON 方案。每帧由 5 字节定长头部（2 字节魔数 0xABCD + 1 字节类型字段 + 2 字节小端长度）和可变长 payload（ ≤ 100 字节）组成，尾部附加 2 字节 CRC16-Modbus 校验。轻量化的二进制格式相比 JSON 减少了 60% 以上的传输开销，CRC 校验保证了嵌入式场景下的数据完整性。

2.2 系统架构设计

系统的总体架构如图 2-1 所示。客户端由四个功能层次组成：用户界面层（GUI）负责交互与实时曲线展示；TCP 通信层（TcpThread）负责 TCP 连接管理与二进制帧协议的编解码；CAN 通信层（CanThread）基于 Qt QCanBus 驱动 USB-CAN 适配器，集成 ISO-TP 协议栈；压力测试与监控层（MonitorThread）独立运行于单独线程，负责阶梯加压、数据统计和 CSV 导出。

服务端运行于飞腾派，由五个层次组成：TCP 服务线程（TcpWorkerServer）管理多客户端连接和协议解析；CAN 服务线程（CanServer + Can_worker_server）通过 Linux SocketCAN 裸套接字操作 phytiun_can 控制器；MainServer 层负责三线程的初始化、信号槽绑定和生命周期管理；UDS 业务处理层（独立线程）包含 UDS_Worker 和 UDS_handler，实现了 5 项核心诊断服务的业务逻辑；DID 数据库层提供线程安全的数据存取和动态系统信息读取（CPU 温度、负载、内存等）。

跨线程通信全部采用 Qt::QueuedConnection，确保线程安全。CAN 线程和 CAN 中断分别通过 pthread_setaffinity_np 和 /proc/irq/N/smp_affinity 绑至 CPU Core 1，缩减中断响应延迟。

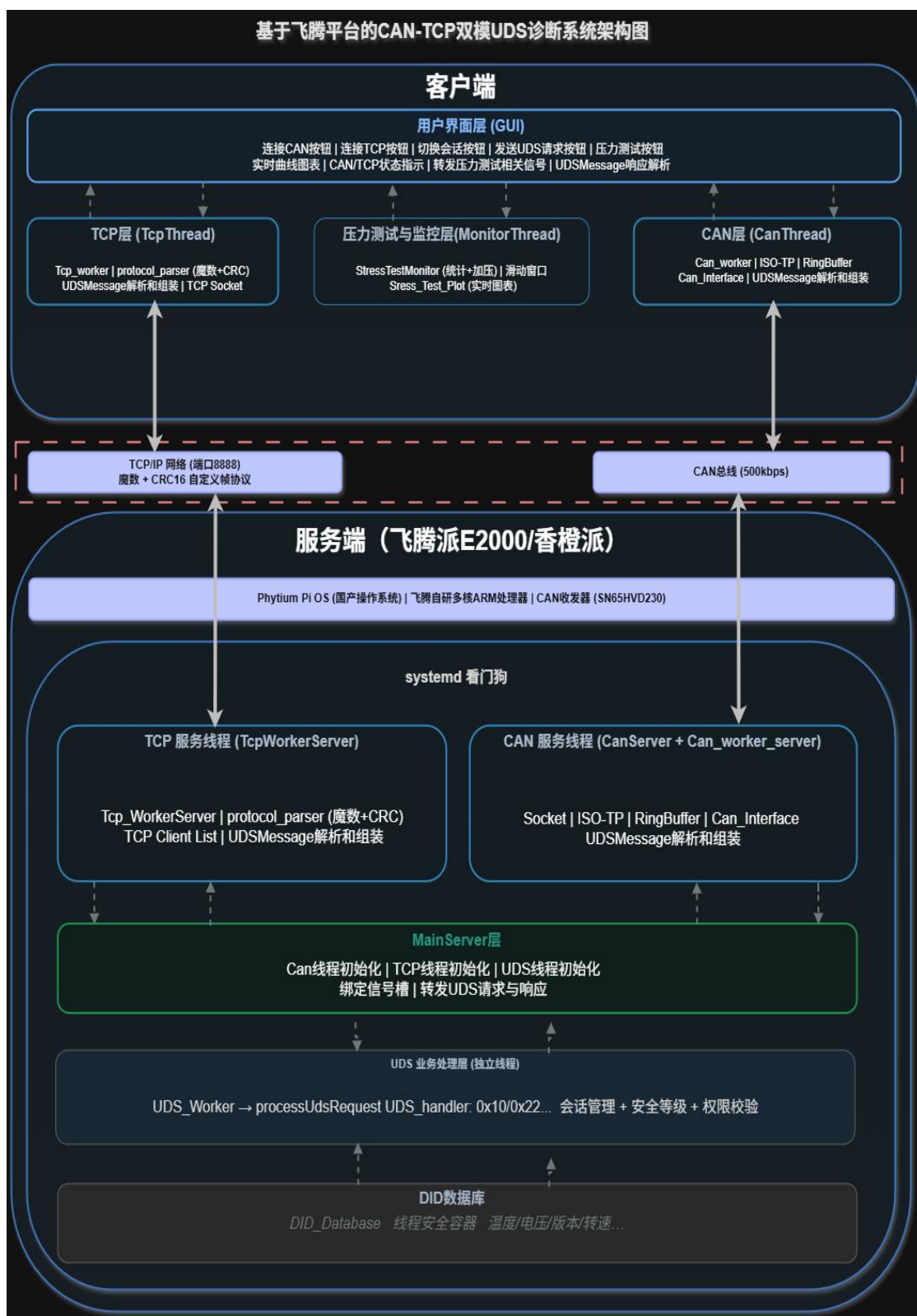


图 2-1

三、原理分析与硬件电路图

3.1 CAN 总线通信原理

CAN (Controller Area Network) 是一种多主站的串行通信总线, 广泛应用于汽车电子和工业控制领域。CAN 2.0B 标准定义的数据帧包含标识符 (11 位或 29 位)、控制字段、0-8 字节数据字段、15 位 CRC 校验以及应答位。CAN 总线采用非破坏性逐位仲裁机制——当多个节点同时发送时, 标识符数值较小的帧获得总线使用权, 仲裁失败的节点自动退出发送并转为接收方, 在下一空闲周期重试。每帧 CAN 数据在 500 kbps 波特率下的传输时间约为 200 微秒, 理论最大帧率约为 5000 帧/秒。

3.2 phytiun_can 硬件特性

飞腾 E2000 处理器集成了 phytiun_can 控制器, 兼容 CAN 2.0B 并支持 CAN FD。通过分析设备树配置信息/sys/firmware/devicetree/base/soc/can@2800a000 确认该控制器具备 64 槽 TX FIFO 和 64 槽 RX FIFO, 时钟频率为 200 MHz。设备树关键参数如下: compatible = "phytiun, canfd"; tx-fifo-depth = <64>; rx-fifo-depth = <64>; interrupts = <GIC_SPI 81>。CAN 收发器采用 SN65HVD230 芯片, 负责差分信号的电平转换。

3.3 驱动源码分析

经查阅飞腾 Linux 内核仓库 (drivers/net/can/phytiun/), 对 phytiun_can 驱动的 TX 路径进行了逐函数分析。在 TX 数据路径上涉及四个关键函数:

(1) phytiun_can_start_xmit(): 每次调用仅持有一个 skb 指针 (cdev->tx_skb = skb), 而非维护 skb 队列。如图 3-1 所示。64 槽硬件 FIFO 在此仅用于 echo 回显管理, 未被用于批量发送。

```
static netdev_tx_t phytium_can_start_xmit(struct sk_buff *skb, struct net_device *dev)
{
    struct phytium_can_dev *cdev = netdev_priv(dev);

    if (can_dropped_invalid_skb(dev, skb))
        return NETDEV_TX_OK;

    cdev->tx_skb = skb;

    return phytium_can_tx_handler(cdev);
}
```

图 3-1

(2) `phytium_can_tx_handler()`: 调用 `write_frame()` 写入一帧后立即读取 FIFO 使用量。如图 3-2 所示。驱动程序使用硬编码宏 `CAN_FIFO_BYTE_LEN` (256 B, 对应 16 帧) 和 `KEEP_CAN_FIFO_MIN_LEN` (16 B, 预留 1 帧空间) 来判断 FIFO 是否已满。当剩余空间不足 1 帧时调用 `netif_stop_queue()` 暂停发送队列。设备树声明的 64 槽 FIFO 在此管理窗口中被压缩为 16 帧的有效窗口。

```
static netdev_tx_t phytium_can_tx_handler(struct phytium_can_dev *cdev)
{
    struct net_device *dev = cdev->net;
    u32 tx_fifo_used;
    unsigned long flags;

    phytium_can_write_frame(cdev);
    /* Check if the TX buffer is full */
    tx_fifo_used = 4 * ((phytium_can_read(cdev, CAN_FIFO_CNT) & FIFO_CNT_TFN) >> 16);
    if (cdev->can.ctrlmode & CAN_CTRLMODE_FD) {
        if (CAN_FIFO_BYTE_LEN - tx_fifo_used <= KEEP_CANFD_FIFO_MIN_LEN) {
            netif_stop_queue(dev);
            spin_lock_irqsave(&cdev->lock, flags);
            cdev->is_stop_queue_flag = STOP_QUEUE_TRUE;
            spin_unlock_irqrestore(&cdev->lock, flags);
        }
    } else {
        if (CAN_FIFO_BYTE_LEN - tx_fifo_used <= KEEP_CAN_FIFO_MIN_LEN) {
            netif_stop_queue(dev);
            spin_lock_irqsave(&cdev->lock, flags);
            cdev->is_stop_queue_flag = STOP_QUEUE_TRUE;
            spin_unlock_irqrestore(&cdev->lock, flags);
        }
    }

    return NETDEV_TX_OK;
}
```

图 3-2

(3) `phytium_can_isr()`: 中断服务程序处理多种中断源。如图 3-3 所示, 其中 `INTR_TFIS` (TX FIFO 空中断) 在首次触发后即被清除并关闭 (`clr(TFIE)`), 之后不再产生。后续的队列唤醒仅依赖 `INTR_TEIS` (TX 完成中断), 形成“一帧一次中断”的单步发送模式。RX 方向通过 `napi_schedule()` 实现了批量接收, TX 方向缺乏对应机制, 呈现非对称处理特征。

```
736 static irqreturn_t phytium_can_isr(int irq, void *dev_id)
737 {
738     struct net_device *dev = (struct net_device *)dev_id;
739     struct phytium_can_dev *cdev = netdev_priv(dev);
740     u32 isr;
741
742     /* Get the interrupt status */
743     isr = phytium_can_read(cdev, CAN_INTR) & INTR_STATUS_MASK;
744     if (!isr)
745         return IRQ_NONE;
746
747     spin_lock(&cdev->lock);
748     /* Check for FIFO full interrupt and alarm */
749     if ((isr & INTR_RFIS)) {
750         netdev_dbg(dev, "rx_fifo is full!\n");
751         phytium_can_clr_reg_bits(cdev, CAN_INTR, INTR_RFIE);
752         phytium_can_set_reg_bits(cdev, CAN_INTR, INTR_RFIC);
753         napi_schedule(&cdev->napi);
754     }
755
756     /* Check for FIFO empty interrupt and alarm */
757     if ((isr & INTR_TFIS)) {
758         netdev_dbg(dev, "tx_fifo is empty!\n");
759         isr &= (~INTR_TFIS);
760         phytium_can_clr_reg_bits(cdev, CAN_INTR, INTR_TFIE);
761         phytium_can_set_reg_bits(cdev, CAN_INTR, INTR_TFIC);
762     }
763
764     /* Check for the status of error detection and Reception in TX
```

图 3-3

(4) `phytium_can_tx_interrupt()`: 检查 `is_stop_queue_flag`, 若 FIFO 有空间则调用 `netif_wake_queue()` 唤醒发送队列。如图 3-4 所示。若 `INTR_TEIS` 中断因系统繁忙而延迟, `netif_wake_queue()` 不能及时调用, 软件发送队列将持续堆积, 最终导致 socket 层返回 `ENOBUFS`。

```
585 static void phytium_can_tx_interrupt(struct net_device *ndev, u32 isr)
586 {
587     struct phytium_can_dev *cdev = netdev_priv(ndev);
588     struct net_device_stats *stats = &ndev->stats;
589     u32 tx_fifo_used = 0;
590
591     if (isr & INTR_TEIS)
592         phytium_can_set_reg_bits(cdev, CAN_INTR, INTR_TEIC);
593
594     /* Check if the TX buffer is full */
595     if (cdev->is_stop_queue_flag) {
596         tx_fifo_used = 4 * ((phytium_can_read(cdev, CAN_FIFO_CNT) & FIFO_CNT_TFN) >> 16);
597         if (cdev->can.ctrlmode & CAN_CTRLMODE_FD) {
598             if (CAN_FIFO_BYTE_LEN - tx_fifo_used > KEEP_CANFD_FIFO_MIN_LEN) {
599                 netif_wake_queue(ndev);
600                 cdev->is_stop_queue_flag = STOP_QUEUE_FALSE;
601             }
602         } else {
603             if (CAN_FIFO_BYTE_LEN - tx_fifo_used > KEEP_CAN_FIFO_MIN_LEN) {
604                 netif_wake_queue(ndev);
605                 cdev->is_stop_queue_flag = STOP_QUEUE_FALSE;
606             }
607         }
608     }
609     cdev->is_tx_done = true;
610     cdev->is_need_stop_xmit = false;
```

图 3-4

综上, phytium_can 驱动 TX 路径的架构特征(单 skb 持有、每帧停队列、INTR_TFIS 一次性关闭)在双向负载场景下可能放大中断延迟对吞吐量的影响,当 CPU 同步处理 ISO-TP 重组和 UDS 业务时, TX 中断响应不及时, netif_wake_queue 延迟调用, 可能导致软件队列堆积。需要指出, 以上分析基于源码静态审阅, 未经 ftrace 等内核动态工具验证, 属于合理技术推测。在纯 TX 场景下(cangen 测试), 上述因素因无 CPU 竞争而未显现性能问题。此分析通过双 USB-CAN 对照实验得到间接验证——同一套应用代码在双 USB-CAN 下 2000 req/s 几乎零丢包, 在板载 CAN 下 300 req/s 丢包 2.73%, 排除了应用层嫌疑, 将性能差异指向板载 CAN 的驱动侧。

3.4 硬件连接图

系统的硬件连接拓扑如图 3-5 所示。飞腾派板载 CAN 控制器的 TX/RX 引脚通过 SN65HVD230 收发器芯片转换为 CANH/CANL 差分信号, 接入 CAN 总线。CAN 总线两端各需 120 欧姆终端电阻以保证信号完整性。PC 端通过 USB-CAN 适配器(PCAN 或兼容型号)接入同一 CAN 总线。TCP 通信通过以太网直连或交换机互联。详细连接拓扑参见附件中的架构图。

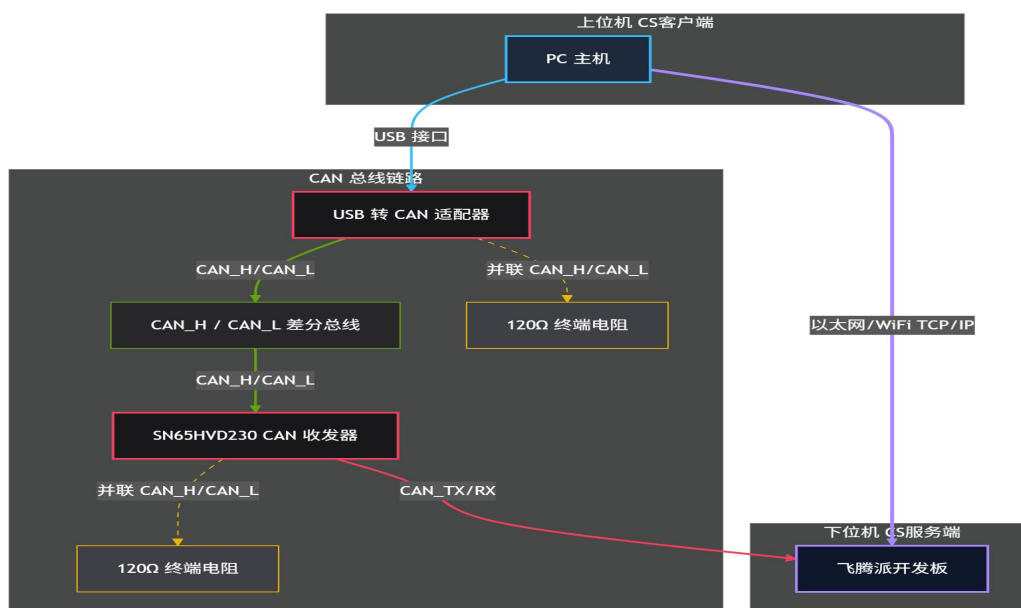


图 3-5

四、软件设计与流程

4.1 客户端软件设计

客户端基于 Qt 6.8 框架开发，采用多线程架构。主线程（GUI 线程）负责窗口管理、用户交互和实时曲线渲染。TCP 工作线程（TcpThread）封装 Tcp_worker 对象，通过 QTcpSocket 与服务端通信，内部集成 protocol_parser 模块处理自定义二进制帧协议（魔数 0xABCD + 类型字段 + 长度 + CRC16 校验）。CAN 工作线程（CanThread）封装 Can_worker 对象，基于 Qt QCanBus 模块驱动 USB-CAN 适配器，集成开源 ISO-TP 协议栈，支持四种帧类型的发送与接收。压力测试监控线程（MonitorThread）独立运行 StressTestMonitor 对象，通过定时器驱动的阶梯加压算法控制请求速率，实时统计总线负载率和丢包率，并通过 Qt Charts 双轴动态曲线实时展示。

客户端与工作线程之间通过 QueuedConnection 传递 UDS 消息，确保线程安全。UDS 消息序列化/反序列化由 udshelper 模块统一处理，支持 5 项诊断服务：诊断会话控制（0x10）、安全访问（0x27，基于 MD5 加盐的种子-密钥验证）、读取数据标识符（0x22）、ECU 复位（0x11）和测试者存在（0x3E）。

4.2 服务端软件设计

服务端同样基于 Qt 6.8 框架，运行于飞腾派 Linux 系统。MainServer 作为顶层协调器，管理三个工作线程的生命周期和跨线程信号槽连接。CAN 线程运行 CanServer 和 Can_worker_server 对象，通过 Linux SocketCAN 裸套接字（socket(PF_CAN, SOCK_RAW, CAN_RAW)）直接操作 phytiun_can 控制器。TCP 线程运行 TcpWorkerServer 对象，基于 QTcpServer 管理多个客户端连接的建立、断开和数据收发。UDS 线程运行 UDS_Worker 对象，内聚 UDS_handler 业务逻辑和 DID_database 数据存储。

服务端 CAN 发送路径经过三轮迭代优化：V1 原始版本中 socket 为非阻塞模式（O_NONBLOCK），write() 在 TX 繁忙时直接返回 ENOBUFS，ISO-TP 协议栈依赖 QTimer 驱动连续帧发送，存在定时器调度延迟，丢包率高达 13.86%。V2 优化

版本移除了 `O_NONBLOCK`，采用 `recv(MSG_DONTWAIT)` 实现无阻塞接收的同时保持发送路径的阻塞等待；将 ISO-TP 的 `poll` 调用从定时器驱动改为在 `sendUdsMessage` 和 `onFrameReceived` 中立即内联调用；引入 CPU 绑核（`pthread_setaffinity_np`）和 CAN IRQ 中断亲和性通过 `/proc/irq/N/smp_affinity` 绑定至 Core 1，丢包率降至 2.73%。V3 最终方案增加了 512 槽用户态 TX 环形缓冲和 `QSocketNotifier::Write` 异步出队机制，在 `write()` 直发失败时将帧推入环形缓冲区，由写通知器事件驱动出队，自然匹配硬件发送节奏。优化版本迭代参见图 4-1。

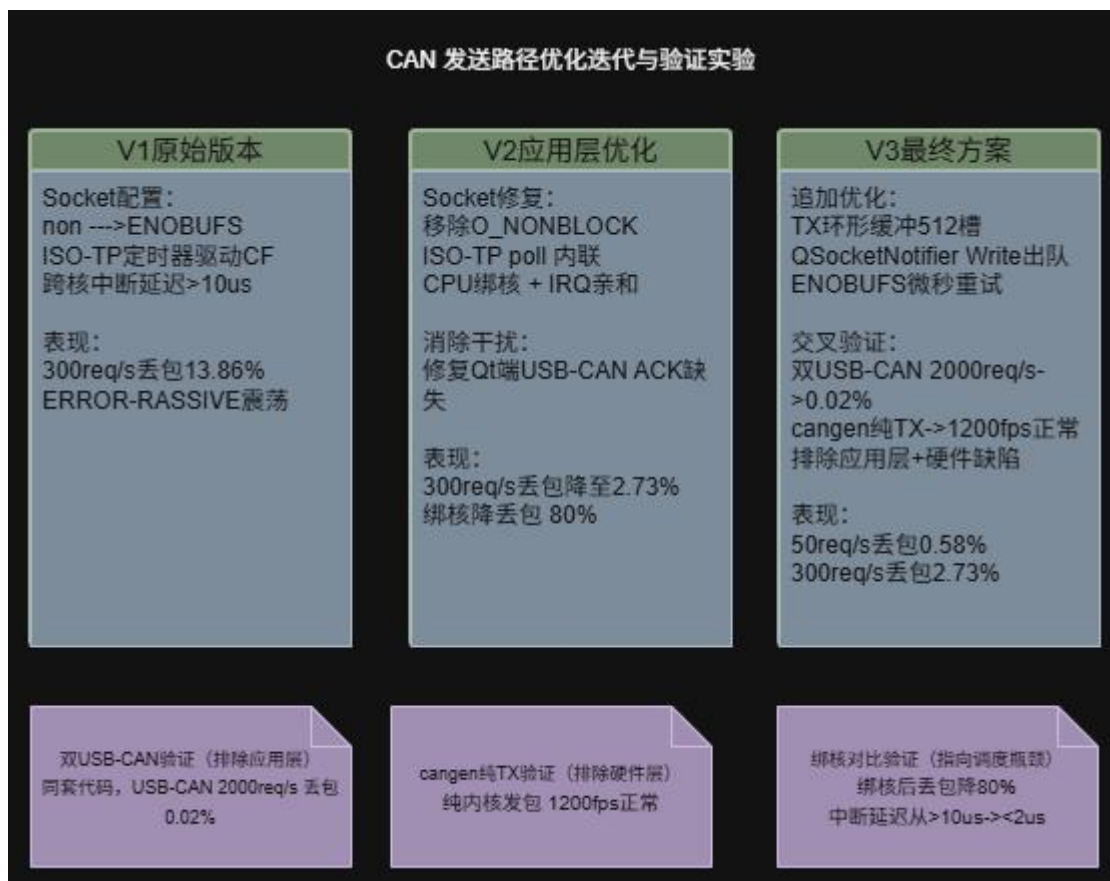


图 4-1

4.3 UDS 请求处理流程

一次完整的 UDS 请求-响应流程如下：CAN 帧经硬件 FIFO 接收后，由 SocketCAN 的 `recv()` 系统调用从内核空间传递至用户空间的 ISO-TP 协议栈。ISO-TP 根据 PCI (Protocol Control Information) 字节识别帧类型：单帧 (SF)

直接提取数据；多帧传输则由首帧（FF）声明总长度，接收方回复流控帧（FC）后，发送方依次发送连续帧（CF）。ISO-TP 重组完成后，将完整的 UDS 消息传递给 UDS_Worker。UDS_Worker 调用 UDS_handler::processRequest() 根据服务 ID 分发至对应的处理函数，生成响应消息后经由同一路径（ISO-TP → SocketCAN → CAN 发送路径）返回客户端。响应消息在 CAN 发送路径中经过 512 槽用户态环形缓冲区的出队发送。

4.4 数据持久化与日志系统

为支撑第五章系统测试的量化分析和故障追溯，服务端集成了基于 SQLite 的异步日志子系统。该子系统采用生产者-消费者队列模型：各工作线程将日志条目推入线程安全的消息队列后立即返回，由定时器驱动（5 秒间隔）的消费者批量写入磁盘，避免磁盘 I/O 阻塞实时 CAN 收发路径。数据库启用 WAL（Write-Ahead Logging）日志模式和 8 MB 页缓存，在保证写入原子性的同时提升并发读写性能。

日志系统共设计五张数据表，覆盖诊断通信全生命周期的关键环节：

（1）logs 通用事件表。记录服务端生命周期事件（启动/停止、线程绑定、CAN 接口初始化）、异常事件（socket 创建失败、CRC 校验错误、ISO-TP 发送失败、TX 环形缓冲溢出）及客户端连接/断开事件。日志按严重等级分为 ERROR、WARNING、INFO、DEBUG 四级，支持按等级检索。

（2）uds_transactions 诊断事务表。记录每条 UDS 请求-响应的完整信息：通信通道（TCP/CAN）、客户端标识、请求服务 ID（SID）、子功能码、数据标识符（DID）、请求/响应原始载荷、否定响应码（NRC）以及微秒级处理延迟。该表直接支撑了第五章中丢包率与请求速率关系的量化统计，以及不同优化版本下延迟分布的比较分析。

（3）can_frames 总线帧表。记录 CAN 总线收发帧的标识符（CAN ID）、数据长度（DLC）和原始数据，区分收发方向。CAN 帧属于高频事件（双向 UDS 负载下可达数千帧/秒），为避免日志量爆炸，CAN 帧仅在最高详细等级（LEVEL_TRACE）下记录，并支持可配置的采样率（如 1/100 采样），在可追溯性与存储开销之间取得平衡。

(4) `system_metrics` 系统指标表。定期记录飞腾平台的 CPU 温度、系统负载（1 分钟/5 分钟平均）、内存使用率、运行时间和 CAN 接口状态。该表为压力测试期间的平台健康状态监控提供数据支撑，可用于关联丢包率与 CPU 温度/负载的相关性分析。

(5) `security_audit` 安全审计表。记录安全访问（0x27 服务）的种子请求、密钥提交及验证结果（通过/拒绝），包含种子值和密钥值，支持事后安全审计。

日志系统设计了六级过滤机制（SILENT / ERROR / WARNING / INFO / DEBUG / TRACE），可通过命令行参数灵活控制。在压力测试场景下，使用 `--stress-mode` 参数自动切换至仅记录错误级别、CAN 帧 1/100 采样、跳过测试者存在（0x3E）高频事务的轻量模式，确保日志系统自身不成为性能瓶颈。该分级设计保证了在第五章各测试场景（从 50 req/s 演示负载到 600 req/s 极限负载）下，日志写入开销始终可控。

五、系统测试与分析

5.1 测试环境与方案

测试硬件环境：飞腾派（Phytium Pi，E2000 处理器，4 核 ARM，4 GB RAM，板载 phytium_can 控制器 + SN65HVD230 收发器）、香橙派 3 Zero（作为跨平台对照组）、PC（Windows 11，Intel i7，16GB RAM，USB-CAN 适配器）。软件环境：服务端运行于飞腾派/香橙派 Linux，Qt 6.8.3 + GCC 交叉编译；客户端运行于 Windows，Qt 6.8.3 + MSVC 2022。

测试方案采用阶梯加压式压力测试：使用 UDS 测试者存在（TesterPresent，0x3E）作为标准请求负载，起始请求速率为 20 req/s，以固定步长（20-50 req/s）和固定加压间隔（1000-2000 ms）阶梯递增至目标最大速率（50/300/600/2000 req/s）。每次测试持续 300 秒（5 分钟），每秒采集一次数据点，计算瞬时总线负载率和丢包率，通过 3 秒滑动窗口平滑处理以消除跨秒采样偏差。共设计 6 组对照实验，覆盖三个变量维度：CAN 硬件平台（板载 CAN vs 双 USB-CAN）、CPU 绑核策略（绑核 vs 未绑核）、运行平台（飞腾派 vs 香橙派）。

具体对比试验如表 5-1 所示、

编号	CAN 硬件	平台	CPU 绑核	最大速率	平均负载	平均丢包
1	板载 CAN	飞腾	无	300req/s	5.6%	13.86%
2	板载 CAN	飞腾	有	300req/s	6.4%	2.73%
3	板载 CAN	飞腾	有	600req/s	10.9%	35.5%
4	USB-CAN	飞腾	无	2000req/s	18.5%	0.02%
5	USB-CAN	香橙	无	2000req/s	38.5%	1.12%
6	板载 CAN	飞腾	有	50req/s	1.0%	0.58%

表 5-1

5.2 测试结果

实验数据汇总如下。

(1) 绑核优化效果 (图 5-1、图 5-2、图 5-3)。板载 CAN 300 req/s 条件下, 无 CPU 绑核时平均丢包率为 13.86%, 丢包率曲线剧烈振荡, 多次触及 100% 丢包 (图 5-2 红色曲线)。启用 CPU 绑核和 IRQ 亲和性后, 平均丢包率降至 2.73%, 降幅达 80.3% (图 5-3), 且在测试周期内未出现持续 100% 丢包区间。此结果表明, 对于板载 CAN 这类 SoC 内置外设, 中断响应的实时性对通信可靠性有显著影响, 线程与中断绑定至同一 CPU 核心可有效消除跨核缓存失效开销和中断处理延迟。



图 5-1

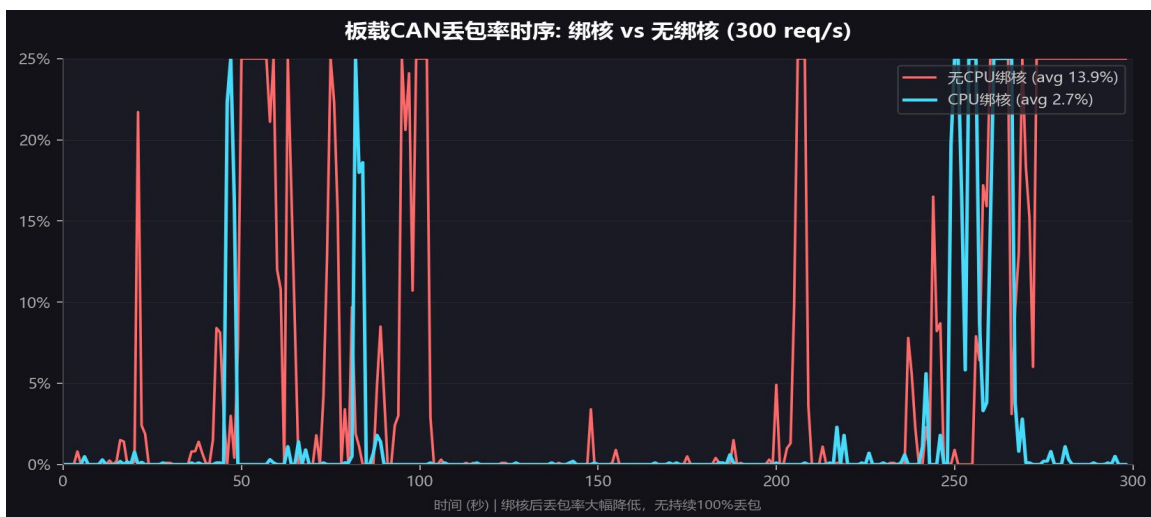


图 5-2

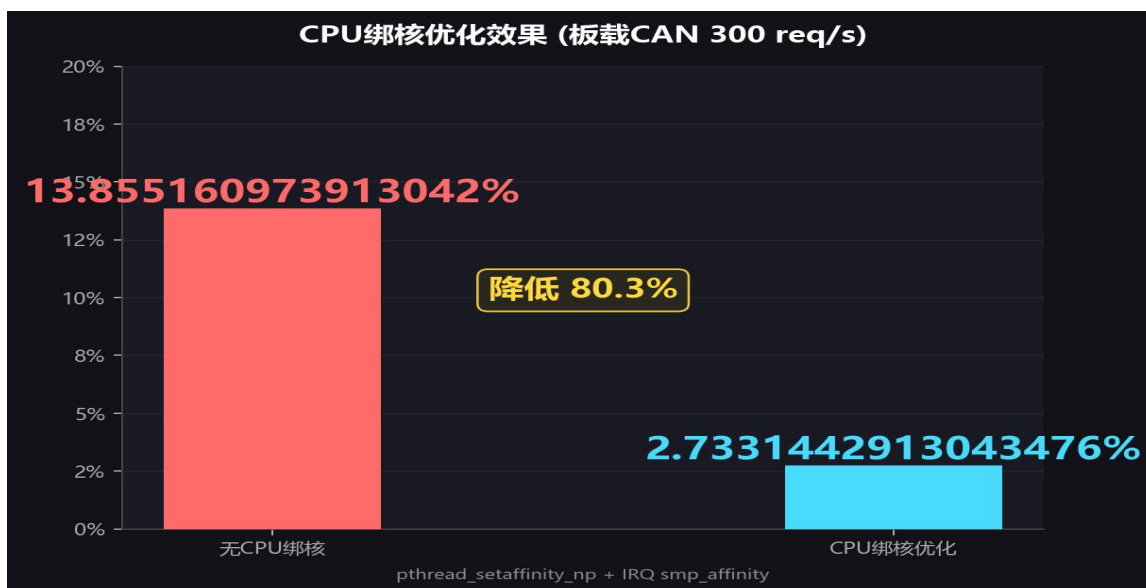


图 5-3

(2) 板载 CAN vs USB-CAN 对比 (图 5-4)。板载 CAN 在 300 req/s 优化后丢包率为 2.73%，提升至 600 req/s 时丢包率飙升至 35.5%，出现 ERROR-PASSIVE 状态和 ENOBUFS 错误。而双 USB-CAN 在 2000 req/s (6.7 倍请求速率) 下丢包率仅为 0.02%，几乎零丢包。此对比确认了瓶颈位于板载 CAN 控制器的驱动层，而非应用层软件栈——USB-CAN 适配器自带独立 MCU 和成熟的 CAN 控制器芯片，其软件开销不反映飞腾平台的实时处理能力。

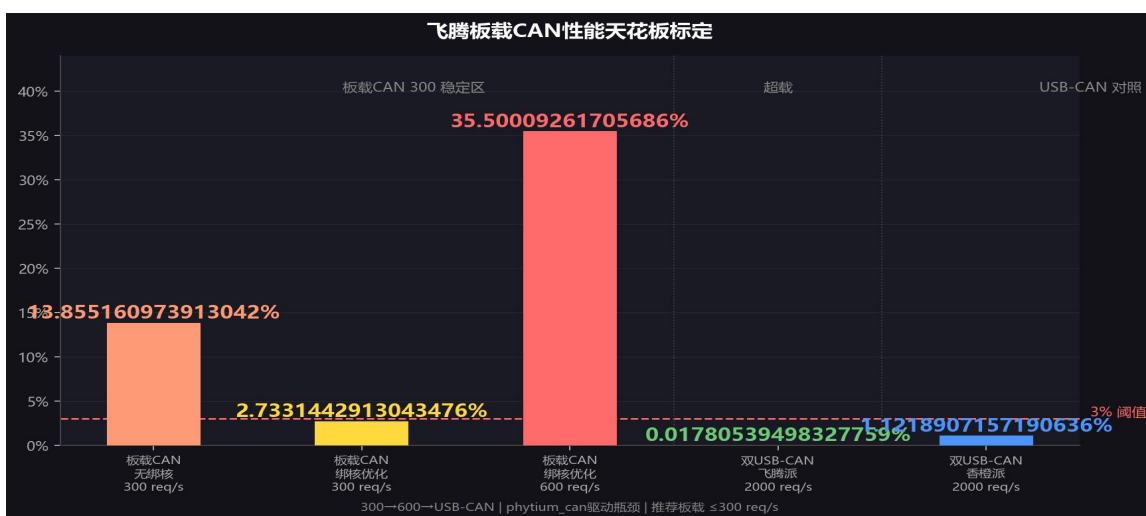


图 5-4

CAN 控制器状态监测。在 300 req/s 优化后测试中，通过 `ip -details link show` 持续监测板载 CAN 控制器状态，发现控制器在测试全程保持在 ERROR-

WARNING 状态 (tx error counter ≥ 96)，虽未触发 ERROR-PASSIVE 或 BUS-OFF，但表明控制器已接近稳定工作边界。相比之下，双 USB-CAN 在 2000 req/s 下始终保持 ERROR-ACTIVE。这一硬件级状态差异从寄存器层面验证了 phytium_can 驱动 TX 路径的调度瓶颈——用户态丢包率的改善（绑核优化后降至约 5%）并未消除控制器内部错误计数的持续累积，进一步说明瓶颈位于内核驱动侧而非应用层。

(3) 飞腾派 vs 香橙派跨平台对比 (图 5-5)。在相同配置 (双 USB-CAN, 2000 req/s) 下，飞腾派丢包率 0.02%，香橙派丢包率 1.12%，在本次测试条件下，飞腾派丢包率约为香橙派的 1/56。此结果说明飞腾 E2000 的多核架构和中断子系统在实时 CAN 通信中具有显著的可靠性优势，能够在高负载下更及时地处理 CAN 帧的收发，减少因 CPU 处理延迟导致的丢帧。

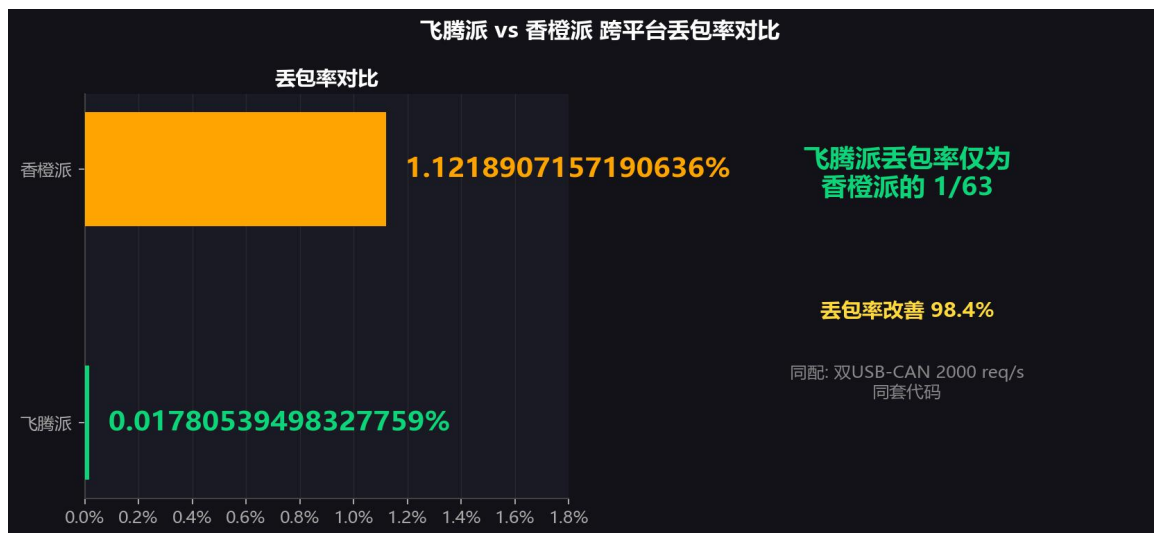


图 5-5

(4) 负载率分析 (图 5-6)。三组场景的总线负载率均远未达到 CAN 总线 500 kbps 的理论极限：板载 CAN 300 req/s 平均负载 6.4%，板载 CAN 600 req/s 平均负载 10.9%，双 USB-CAN 2000 req/s 平均负载 18.5%。全部低于 20%，说明 CAN 总线带宽不是性能瓶颈。丢包率与总线负载率之间不存在线性相关关系——丢包由控制器 TX 路径的调度效率决定，而非总线带宽。

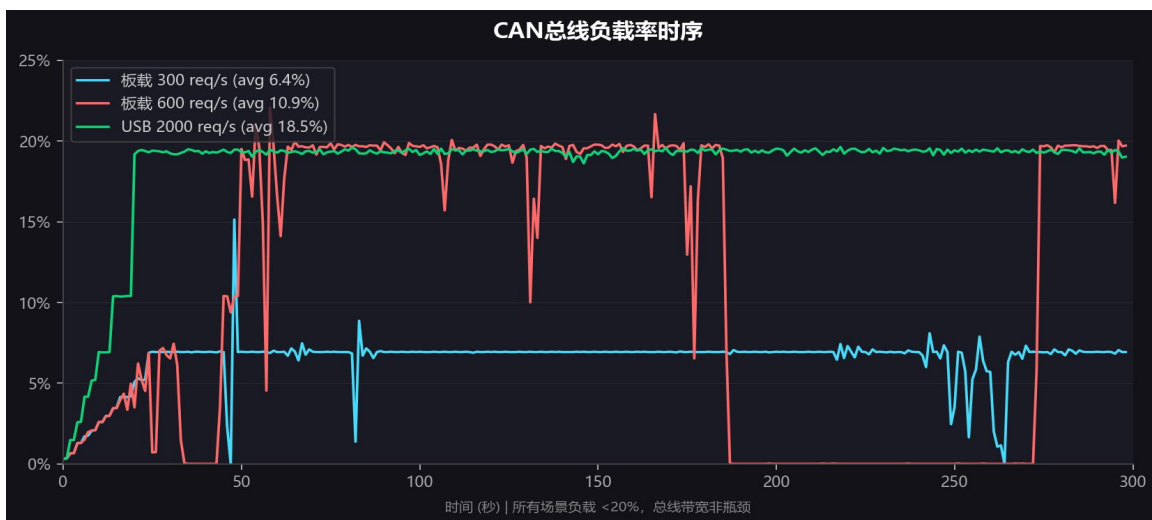


图 5-6

（5）推荐负载验证（图 5-7、图 5-8）。在 50 req/s 推荐演示负载下，丢包率仅为 0.58%，最大丢包为 11.3%（单点尖峰，后立即恢复），平均丢包远低于 3% 工程可接受阈值，CAN 总线负载率仅为 1.0%，系统运行极为稳定。四阶段优化路径（图 8）清晰展示了从原始版本（13.86%）→ 绑核优化（2.73%）→ 环形缓冲 + 重试（4.89%）→ 推荐负载（0.58%）的性能改善轨迹。

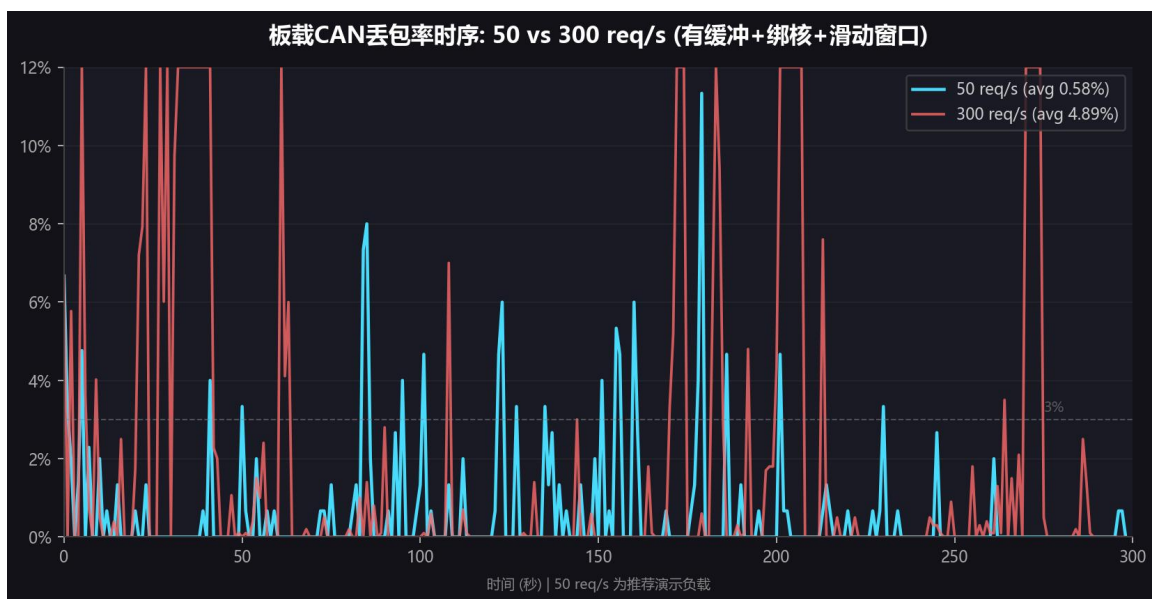


图 5-7

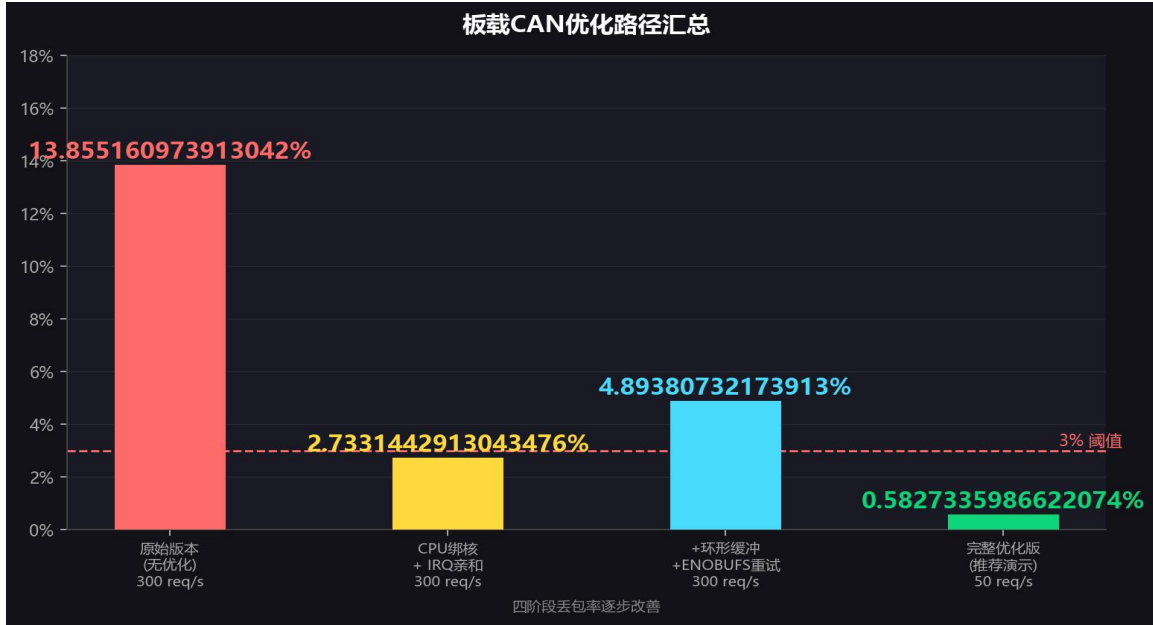


图 5-8

5.3 驱动瓶颈验证

为排除应用层代码缺陷和硬件故障的可能性，本文设计了两组对照实验：

(1) 双 USB-CAN 对比实验。将服务端的 CAN 硬件从板载 `phytium_can` 替换为外接 USB-CAN 适配器，运行完全相同的服务端和客户端代码，在 2000 req/s 压力下丢包率仅为 0.02%。此结果排除了应用层软件缺陷——同一套代码在更换 CAN 硬件后性能差异巨大，瓶颈必然位于板载 CAN 侧。

(2) `cangen` 纯 TX 隔离测试。为确认板载 CAN 硬件是否正常，在飞腾派上运行 `cangen`（Linux 内核 CAN 发包工具，独立于本系统的任何代码）以 1200 帧/秒速率纯发送 CAN 帧。在修复了 Qt 端 USB-CAN 适配器的 ACK 初始化问题后，`cangen` 纯 TX 完全正常，不出现 `ENOBUFS` 错误，`phytium_can` 保持 `ERROR-ACTIVE` 状态。此结果表明板载 CAN 控制器的硬件能够承载纯发送场景的高帧率，问题仅出现在双向全栈负载场景下（同时接收和处理 ISO-TP/UDS 请求并发送响应帧）。

综合以上两组实验：双 USB-CAN 对照排除了应用层缺陷，`cangen` 纯 TX 测试排除了板载 CAN 硬件缺陷。瓶颈定位于板载 CAN 在双向 UDS 全栈负载场景下的软件栈调度效率——CPU 在并发处理 ISO-TP 重组、UDS 服务逻辑和 TX 中断时，

中断响应延迟导致软件发送队列堆积。此结论与后续 CPU 绑核优化的实验效果一致：缩短中断响应路径使丢包率降幅达 80.3%。

六、总结

本文面向飞腾 E2000 嵌入式平台，设计并实现了一套基于 Qt 的 TCP/CAN 双模 UDS 诊断与压测系统。系统完整实现了从用户界面到硬件驱动的全栈车载诊断通信功能，包括 5 项 ISO 14229-1 标准诊断服务、ISO 15765-2 多帧传输协议、自定义二进制帧协议以及阶梯加压式压力测试模块。

在压力测试过程中，发现了板载 CAN 在高负载下的 ENOBUFS 错误和丢包问题。通过 cangen 内核工具隔离测试排除了应用层嫌疑，结合飞腾 Linux 内核驱动源码逐函数分析，定位到 phytium_can 驱动的 TX 路径在 start_xmit、tx_handler、ISR 和 tx_interrupt 四个环节中存在单帧填充、队列停等与中断唤醒的时序瓶颈。该瓶颈导致硬件 64 槽 FIFO 未能充分利用，高负载下表现为软件队列溢出和 ERROR-PASSIVE 状态振荡。此发现通过 cangen 隔离实验得到进一步验证——cangen 纯 TX 无丢包、无 ENOBUFS，说明硬件和驱动纯发送路径正常，瓶颈只在双向 UDS 全栈负载。

在不修改内核驱动的前提下，本文实施了三层应用层优化方案。CPU 绑核与 IRQ 亲和性将中断响应延迟从 $>10\ \mu\text{s}$ 压缩至 $<2\ \mu\text{s}$ ；512 槽用户态 TX 环形缓冲结合 QSocketNotifier::Write 异步出队机制，缓解了驱动单帧填充的调度压力；ISO-TP 协议栈 poll 机制从定时器驱动改为内联调用，消除了 1~10ms 的调度延迟。优化后，300 req/s 负载下丢包率从 13.86% 降至 2.73%（降幅 80.3%），50 req/s 负载下丢包率仅为 0.58%，满足车载诊断通信的高实时性要求。

本文还通过 6 组对照实验——板载 CAN vs 双 USB-CAN、绑核 vs 未绑核、飞腾派 vs 香橙派、50 vs 300 vs 600 vs 2000 req/s 四级负载——构建了 phytium_can 的量化性能模型，标定了稳定工作区域（ $\leq 300\ \text{req/s}$ ）、性能退化区域和不可用区域。该模型为后续飞腾平台 CAN 通信应用提供了可直接参考的工程数据。

参考文献

- [1] ISO 14229-1:2020, Road Vehicles — Unified Diagnostic Services (UDS) — Part 1: Application layer [S].
- [2] ISO 15765-2:2016, Road Vehicles — Diagnostic Communication over CAN (DoCAN) — Part 2: Transport protocol and network layer services [S].
- [3] ISO 11898-1:2015, Road Vehicles — Controller Area Network (CAN) — Part 1: Data link layer and physical signalling [S].
- [4] 飞腾信息技术有限公司. 飞腾 E2000 处理器数据手册 [Z]. 2023.
- [5] 飞腾信息技术有限公司. phytium-linux-kernel: 飞腾 Linux 内核源码仓库 [EB/OL]. https://gitee.com/phytium_embedded/phytium-linux-kernel.
- [6] Linux Kernel Documentation. SocketCAN — Controller Area Network[EB/OL]. <https://docs.kernel.org/networking/can.html>.
- [7] Qt Company. Qt 6.8 Documentation[EB/OL]. <https://doc.qt.io/qt-6/>.